中国海洋大学

机器人集成小组项目

实验报告

基于 YOLO11n 的桌面物体检测及 ROS2 结果发布实验

学生姓名：李书宇

学　　号：24020036035

学院专业：信息科学与工程学部计算机学院

2024 级计算机科学与技术（中外合作办学）

课程名称：机器人集成小组项目

指导教师：周小伟

学校名称：中国海洋大学

2026 年 9 月

摘 要

针对机器人桌面场景中的物体定位与检测结果发布需求，本实验实现了基于YOLO11n 的五类目标检测程序。数据集共 5000 张图像、8507 个目标框，其中 1000 张自主采集来源样本由 10 张独立实拍母图扩展得到，包含 10 张尺寸统一的基准版本和 990 张确定性增强图像，其余 4000 张选自 Open Images V7。自主采集部分按母图分组划分，避免同源增强图像跨集合交叉。模型以 640 像素输入微调 12 轮，在 650 张测试图像上取得 0.8385 的 Precision、0.7697 的 Recall、0.8386 的 mAP@0.5 和 0.5735 的 mAP@0.5:0.95。20 张单目标样本按最高置信度类别判定，19 张预测正确，正确率为 95%。在 Jetson Orin NX 上，PyTorch FP16 模型的平均 `model.predict` 调用耗时为 25.951 ms，对应 38.534 FPS；ROS2 结果主题的实测发布频率为 29.99 Hz。程序能够处理图片、视频和摄像头输入，保存检测记录与错误样本，并通过 ROS2 发布结构化结果和带框图像。

关键词：目标检测；YOLO11n；桌面物体；数据增强；Jetson Orin NX；ROS2

Abstract

Desktop manipulation requires object categories and locations to be available to downstream robot software. This experiment implements a YOLO11n detector for mouse, keyboard, laptop, cup, and headphones, together with a ROS2 result-publishing interface. The dataset contains 5,000 images and 8,507 bounding boxes. The 1,000-image subset derived from self-collected material originated from 10 independently captured parent photographs and contains 10 size-normalized base images and 990 deterministic augmentations; the remaining 4,000 images were selected from Open Images V7. Derivatives of the same parent photograph were assigned to a single split to prevent source overlap across training and evaluation data. After 12 epochs of fine-tuning at an input size of 640 pixels, the model achieved a precision of 0.8385, a recall of 0.7697, an mAP at IoU 0.5 of 0.8386, and an mAP over IoU 0.5–0.95 of 0.5735 on 650 test images. The highest-confidence predicted class was correct for 19 of 20 additional single-object samples. On a Jetson Orin NX, the PyTorch FP16 model achieved a mean `model.predict` call time of 25.951 ms, corresponding to 38.534 FPS, while the ROS2 result topic was observed at 29.99 Hz. The program processes images, videos, and camera streams, stores detection records and error cases, and publishes structured detections and annotated images through ROS2.

Keywords: object detection; YOLO11n; desktop objects; data augmentation; Jetson Orin NX; ROS2

1 引言

桌面抓取和人机交互不仅需要判断物体类别，还需要获得目标在画面中的位置。鼠标、键盘和笔记本电脑经常在有限区域内同时出现；笔记本打开时，整机与可见键盘又形成相互包含的目标。光照、遮挡和拍摄角度变化会进一步改变目标外观。检测程序因而需要连续处理图像，并同步输出类别、边界框和置信度。

YOLO 在一次前向传播中完成类别预测与边界框回归，属于单阶段检测方法 [1]；SSD 则利用多尺度特征图处理尺寸差异较大的目标 [2]。两类方法均省去了独立的候选区域生成阶段，适合对连续图像进行快速推理。本实验据此选用规模较小的 YOLO11n，并保持网络结构不变。

实验围绕两项任务展开：一是在母图级划分控制下构建混合来源数据集，并评价五类桌面物体的检测性能；二是将同一推理流程用于结果可视化、记录保存和 ROS2 消息发布。三项评价分别对应测试集检测、单目标类别判定和摄像头预测调用性能，ROS2 接口复用同一检测结果结构。

2 相关工作

2.1 单阶段目标检测

原始 YOLO 直接从整幅图像预测边界框和类别概率 [1]。SSD 在多层特征图上设置默认框，使不同尺度的目标能够在相应分辨率上完成检测 [2]；Focal Loss 则从损失加权角度缓解稠密检测中的前景、背景样本不平衡 [3]。这些方法分别从整体回归、多尺度预测和损失重加权三个角度发展了单阶段检测。

数据增强可在不改变部署网络的情况下扩展训练样本外观。YOLOv4 将颜色变化、几何变换与 Mosaic 组合用于目标检测训练 [4]。为避免几何变换明显偏离本实验拍摄场景，离线扩展限制旋转幅度，并与训练阶段的在线增强分开设置。

YOLO11 由 Ultralytics 软件包提供，训练、验证、预测与模型导出使用统一接口，其中 n 规模模型面向轻量推理 [5]。本文以官方代码仓库中的软件版本为准，早期 YOLO 文献仅用于说明其单阶段检测技术脉络。

2.2 数据资源、评价方法与通信框架

Open Images 提供自然场景图像及多种标注，其目标检测子集包含大量多目标画面 [6]。实验使用 V7 版本中与任务对应的五类图片 [7]，并将公开边界框转换为 YOLO 格式。评价采用 Precision、Recall、mAP@0.5 和 mAP@0.5:0.95；后者沿用 COCO 的多 IoU 阈值计算方式 [8]，各项数值由 Ultralytics 验证接口计算 [9]。

ROS2 通过主题传递传感器数据和机器人状态。发布者与订阅者约定主题名称和消息类型后即可交换数据，无需直接调用对方程序 [10]。这种通信方式允许检测节点独立运行，下游模块按需订阅结构化目标信息或可视化图像。

3 数据集构建

3.1 类别与标注规则

检测类别依次为鼠标（mouse）、键盘（keyboard）、笔记本电脑（laptop）、杯子（cup）和头戴式耳机（headphones），类别编号为 0 至 4。标签采用 Ultralytics YOLO 水平边界框格式，每行记录类别编号、归一化中心坐标及归一化宽高 [11]。打开的笔记本电脑以整机区域标注为 laptop，可见键盘区域另行标注为 keyboard；目标受到遮挡时，仅框选可见部分。公开数据沿用 Open Images 边界框，在构建阶段完成类别映射、格式转换和坐标合法性检查。

3.2 来源构成与离线扩展

数据集包含 1000 张自主采集来源样本和 4000 张 Open Images V7 图片 [7]。自主采集部分以 10 张独立实拍母图为基础，每张母图形成 1 张仅统一至 1280×960 像素的基准版本和 99 张确定性增强图像。增强参数由随机种子 20260831 及母图名称、版本号共同确定，包含 $[-9°, 9°]$ 旋转、$[0.88, 1.12]$ 尺度变换、水平或垂直方向不超过图像尺寸 7% 的平移、概率为 0.5 的水平翻转，以及亮度、对比度、模糊和噪声扰动。母图哈希与每个版本的变换参数均写入来源记录。图1给出了四组基准版本及其增强结果。



图 1 自主采集来源素材示例。上排（a）–（d）为四张母图的尺度统一版本，下排（e）–（h）为对应的确定性增强版本；边界框随几何变换同步更新。

3.3 数据划分与质量检查

10 张母图按 8:1:1 分配至训练、验证和测试集合，同一母图的全部版本仅进入一个集合。公开图片保留 Open Images 的原始来源划分，并在各划分内按稳定哈希抽样。最终训练集、验证集和测试集分别包含 4000、350 和 650 张图像，来源构成及目标框数量见表1。5000 张图像均有同名标签文件，共计 8507 个目标框。

表 1 数据划分、来源构成与目标框统计

划分	自采来源	公开数据	图像合计	目标框
Train	800	3200	4000	6773
Validation	100	250	350	627
Test	100	550	650	1107
合计	1000	4000	5000	8507

数据质量控制涵盖图像可读性、图像与标签配对、类别编号、归一化坐标、重复来源 ID、完全重复文件及母图跨集合情况；检查结果中未出现损坏图像、缺失标签或跨集合母图。五类目标框分布见表2。laptop 共 2549 个，占 29.96%；headphones 为 871 个，占 10.24%。类别数量并不均衡，因此评价除总体指标外还需比较逐类结果。

表 2 五类目标框分布

类别	目标框数	比例（%）
mouse	1724	20.27
keyboard	1966	23.11
laptop	2549	29.96
cup	1397	16.42
headphones	871	10.24
合计	8507	100.00

4 检测系统与实现

4.1 总体流程

系统由数据构建、模型训练、通用推理和 ROS2 发布四部分组成，处理流程见图2。训练入口读取数据配置并生成模型权重；推理入口接收图片、视频或 USB 摄像头画面，输出检测记录、带框结果和错误样本；ROS2 节点复用相同的模型与阈值配置，将检测结果封装为消息。训练、推理和通信模块共享类别映射及运行参数，并分别保留独立入口。

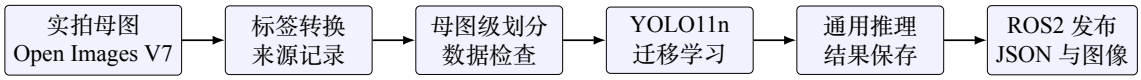


图 2 数据构建、模型训练、推理与 ROS2 结果发布流程。

4.2 模型微调配置

模型以先前训练的五类 YOLO11n 权重 `models/expanded_v1/best.pt` 初始化，该检查点与本实验采用相同的类别顺序。正式训练共 12 轮，批量大小为 32，输入

尺寸为 640，数据加载进程数为 0，早停耐心值为 4。训练在 MPS 后端执行，随机种子为 0，并启用确定性计算和自动混合精度。优化器采用 Ultralytics 的自动选择策略，具体运行参数由训练目录中的 `args.yaml` 保存。

在线增强参数包括 0.015 的色相变化、0.7 的饱和度变化和 0.4 的明度变化，旋转、平移与缩放参数分别为 5 度、0.1 和 0.3，水平翻转概率为 0.5。Mosaic 的初始概率为 0.5，并在最后 10 轮关闭；MixUp 未启用。每轮结束后在验证集上计算指标，训练目录同时保存最后一轮权重与验证指标最优权重。

4.3 推理记录与 ROS2 接口

推理程序以 640 为输入尺寸参数，候选框经置信度筛选和非极大值抑制得到最终检测。每条记录包含类别编号、类别名称、置信度以及像素坐标边界框 (x_1, y_1, x_2, y_2)，结果同时用于绘制检测框和写入 CSV。图片、视频、摄像头输入共用同一结果结构。

检测程序另封装为 ROS2 节点。摄像头模式由节点直接读取 USB 视频帧，主题模式则订阅 `/camera/image_raw`。每帧推理后，`/object_detection/result` 以 `std_msgs/String` 承载 JSON，字段包括时间戳、帧号、按预测调用速率计算的指数平滑 FPS 和检测列表；`/object_detection/image` 以 `sensor_msgs/Image` 发布带框图像。数值结果与图像分开传输，使控制程序和可视化程序能够按需订阅同一检测节点 [10]。

4.4 Jetson 部署实现

训练所得验收权重 `best.pt` 部署于 Jetson Orin NX。实机运行采用 NVIDIA 版 PyTorch 2.5.0、CUDA GPU 设备 0 和 FP16 精度，模型输入固定为 640×640，confidence 与 NMS IoU 阈值分别为 0.50 和 0.45。USB 摄像头 `/dev/video0` 的实际取帧分辨率为 640×480，检测节点在送入网络前由 Ultralytics 完成尺度变换和预处理。

该设备已安装 TensorRT 10.7.0.23 运行库，但当前 Python 环境未安装 `tensorrt` binding，因而本次实测使用 `best.pt` 的 PyTorch FP16 后端，而非 TensorRT 序列化引擎。TensorRT 引擎通常需要针对目标 GPU 与运行环境构建 [12, 13]；本实验将已安装运行库与实际推理后端分开记录，避免把软件依赖的存在误写为 TensorRT 性能结果。

5 实验设置与评价协议

5.1 训练与离线评价环境

训练与软件测试在 macOS、Apple M4 处理器上完成，计算后端采用 MPS。Python、PyTorch 和 Ultralytics 版本分别为 3.12.4、2.13.0 和 8.4.133。第 12 轮权重在验证集上的 Precision、Recall、mAP@0.5 和 mAP@0.5:0.95 分别为 0.7407、0.7512、0.7334 和 0.4621；其中 mAP@0.5:0.95 为 12 轮最高值，故选作后续测试模型。

5.2 Jetson 实机环境与运行配置

边缘端测试使用 NVIDIA Jetson Orin NX Engineering Reference Developer Kit Super。设备保持 MAXN_SUPER 功耗模式并采用动态调频，未启用 jetson_clocks 的固定频率覆盖。实机环境、模型后端和输入设置见表3。

表 3 Jetson 实机环境与运行配置

字段	实测或已核对值
设备与内存	NVIDIA Jetson Orin NX Engineering Reference Developer Kit Super；16 GB，系统可见 16031376 kB（约 15.29 GiB）
系统软件	JetPack 6.2.1；L4T 36.4.4；CUDA runtime 12.6.77；TensorRT libraries 10.7.0.23；ROS2 Humble
功耗配置	nvpmodel: MAXN_SUPER，mode 0；动态调频，FreqOverride=0
模型与后端	models/experiment1_1000self_4000public_v1/best.pt；YOLO11n；NVIDIA build PyTorch 2.5.0；CUDA GPU device 0
计算精度	FP16
输入配置	模型输入 640 × 640；HDR CAMERA-A USB 摄像头 /dev/video0；实际取帧 640 × 480
检测阈值	confidence 0.50；NMS IoU 0.45

5.3 评价指标

设真正例、假正例和假负例分别为 TP、FP 和 FN，Precision 与 Recall 定义为

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}. \tag{1}$$

预测框与真实框的交并比为

$$\mathrm{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \tag{2}$$

其中 B_p 和 B_g 分别表示预测框与真实框。mAP@0.5 在 IoU 阈值 0.5 下计算各类别 AP 的均值；mAP@0.5:0.95 则在 0.5 至 0.95、步长 0.05 的多个阈值上平均，对边界框定位提出更严格要求 [8, 9]。

5.4 测试协议

测试由三项协议组成。首先，650 张测试图像通过 Ultralytics 8.4.133 的 model.val 接口评价，参数为 split=test、imgsz=640、batch=16 和 device=mps，其余阈值沿用该版本的验证默认配置。该部分输出总体及逐类指标，并生成归一化混淆矩阵。

其次，从已保存的单目标图片中每类选取 4 张，共 20 张，以 0.5 置信度阈值和 0.45 NMS IoU 阈值进行预测。图像中最高置信度检测框的类别与预设类别一致时，记为类别判定正确；未检出目标或最高置信度类别不一致均记为错误。该正确率仅用于 20 样

本验收，不与基于 IoU 匹配的测试集检测指标混用。

最后，在 Jetson Orin NX 上使用 USB 摄像头测量实时性能。计时点设在摄像头完成取帧之后，先处理 20 帧用于预热，再统计后续 200 帧。每帧计时覆盖 Ultralytics `model.predict` 内部的预处理、GPU 推理与后处理，不包括摄像头读取、结果绘制、视频或日志保存以及 ROS2 消息发布。模型采用 PyTorch FP16 后端，输入尺寸、confidence 与 NMS IoU 阈值分别为 640、0.50 和 0.45。

ROS2 通信性能通过 `ros2 topic hz` 对 `/object_detection/result` 独立测量，滑动窗口包含 200 条消息，采样时间为 15 s。该频率表示结果消息到达订阅端的实际速率，与节点 JSON 中按 `model.predict` 调用耗时计算的指数平滑 FPS 含义不同。两项结果分别用于描述模型预测调用性能和检测节点的实际结果发布能力。逐帧 benchmark 记录保存于 Jetson 端的 `outputs/benchmark/jetson_orin_nx_camera.csv`。

6 实验结果与分析

6.1 训练过程与总体检测结果

图3记录了 12 轮训练。三项训练损失总体下降，验证集损失存在波动；验证集 mAP@0.5 与 mAP@0.5:0.95 在第 12 轮分别达到 0.7334 和 0.4621。第 12 轮两项 mAP 仍在上升，本次 12 轮训练内未出现明显的平台区间。

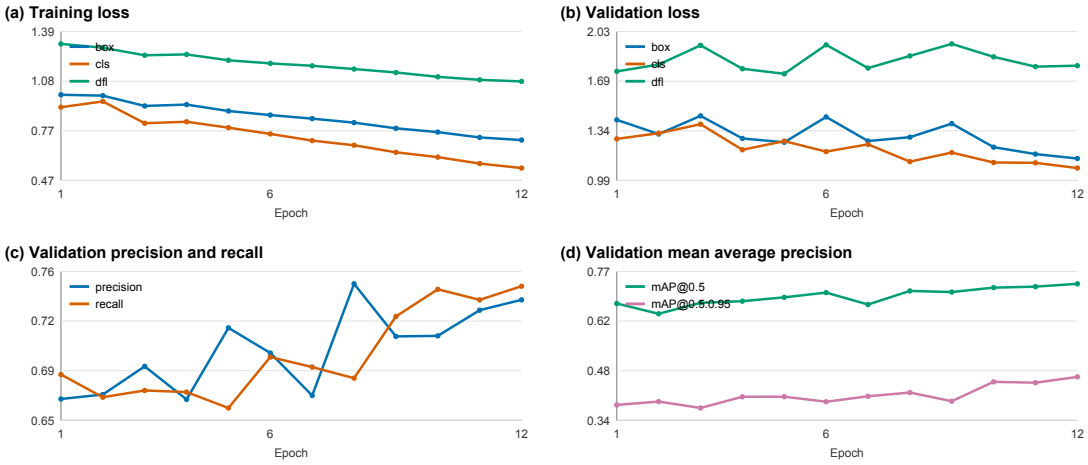


图 3 YOLO11n 微调过程。(a) 训练损失；(b) 验证损失；(c) 验证集 Precision 与 Recall；(d) 验证集平均精度。

650 张测试图像的检测结果见表4。总体 Precision 为 0.8385，Recall 为 0.7697，后者表明模型对真实目标的覆盖程度低于已保留预测的准确程度。mAP@0.5 为 0.8386，多阈值平均的 mAP@0.5:0.95 为 0.5735，两者相差 0.2651，说明模型在更严格的 IoU 标准下综合检测性能明显下降，边界定位质量对评价阈值较为敏感。

表 4　测试集总体与逐类检测指标

类别	P	R	AP_{50}	$AP_{50:95}$
mouse	0.9325	0.9101	0.9501	0.7546
keyboard	0.8346	0.8469	0.9012	0.6361
laptop	0.9121	0.7529	0.8640	0.4599
cup	0.9091	0.7409	0.8571	0.6751
headphones	0.6041	0.5978	0.6203	0.3419
总体	0.8385	0.7697	0.8386	0.5735

6.2　类别差异与错误表现

mouse 的结果最好，AP_{50} 与 Recall 分别为 0.9501 和 0.9101。keyboard 同时包含独立键盘和笔记本电脑的可见键盘区域，其 AP_{50} 仍达到 0.9012。laptop 的 Precision 为 0.9121，但 Recall 为 0.7529，$AP_{50:95}$ 进一步降至 0.4599，指标组合显示其目标覆盖和严格 IoU 条件下的定位表现相对较弱。

headphones 的 Precision、Recall 和 AP_{50} 分别为 0.6041、0.5978 和 0.6203，均为五类最低。该类目标框数量最少，样本中还存在展开、折叠及局部遮挡等外观差异。现有数据只能说明较少的样本量与较低指标同时出现，不能据此认定直接因果关系。

测试集归一化混淆矩阵见图4。按真实类别归一化后，mouse、keyboard、laptop、cup 和 headphones 的对角线值依次为 0.93、0.89、0.82、0.78 和 0.67。背景行中 cup 与 headphones 的数值分别为 0.22 和 0.32，说明未与有效预测框匹配的真值目标在这两类中更为常见；显式类别间的非对角混淆相对较少。这一分布与 cup 和 headphones 相对较低的 Recall 一致。



图 4　650 张测试图像上的归一化混淆矩阵。列为真实类别，行为预测类别。

保存结果中的代表性 laptop 错误表现为目标未检出或边界框偏移。由于本实验未对遮挡、视角和画面位置等错误因素逐项计数，这些图像仅作为典型案例保存，不用于推断各因素的发生比例。

6.3 20 样本类别判定

20 张单目标样本中，最高置信度预测类别与预设类别一致的样本为 19 张，正确率为 95%，高于实验要求的 80%。唯一错误样本的预设类别为 laptop，在 0.5 置信度阈值下没有生成有效检测框。该样本已记录为漏检，并单独保存错误图像。

6.4 Jetson 实时检测与 ROS2 发布性能

Jetson 实机在 20 帧预热后统计 200 个摄像头帧，`model.predict` 平均调用耗时为 25.951 ms，对应 38.534 FPS；最低逐帧预测调用速率为 37.952 FPS。Ultralytics 记录的平均网络推理耗时为 20.575 ms，最大网络推理耗时为 20.896 ms。调用耗时比网络推理耗时高 5.376 ms，该差值包含预处理、后处理和预测接口调用开销。平均预测调用速率约为 5 FPS 验收阈值的 7.71 倍，最低逐帧速率亦高于该阈值。

`/object_detection/result` 在 15 s 测量窗口中的实际发布频率为 29.99 Hz，与约 30 FPS 的摄像头输入速率接近。该频率低于独立 benchmark 得到的模型预测调用速率，因为连续节点运行还涉及取帧、结果封装、绘框及图像消息处理。两项测量均超过 5 FPS，表明模型推理与 ROS2 结果发布在当前配置下均满足实时性要求。主要检测与实机结果汇总于表5。

表 5 主要检测结果与 Jetson 实机性能汇总

测试项目	结果
测试集规模	650 张，1107 个目标框
总体 mAP@0.5 / mAP@0.5:0.95	0.8386 / 0.5735
20 样本类别判定正确率	19/20，95%
Jetson 推理后端	PyTorch FP16，CUDA device 0
Jetson 预热/计时帧数	20/200
平均网络推理耗时	20.575 ms
平均 `model.predict` 调用耗时	25.951 ms
平均/最低预测调用速率	38.534/37.952 FPS
最大网络推理耗时	20.896 ms
ROS2 结果主题频率	29.99 Hz（15 s，window 200）
实时性要求与判定	$\geq$ 5 FPS，PASS

参考文献

[1] REDMON J, DIVVALA S, GIRSHICK R, et al. You only look once: Unified, real-time object detection[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 779-788. DOI: 10.1109/CVPR.2016.91.

[2] LIU W, ANGUELOV D, ERHAN D, et al. SSD: Single shot MultiBox detector[C/OL]//Lecture Notes in Computer Science: Vol. 9905 Computer Vision – ECCV 2016. 2016: 21-37. DOI: 10.1007/978-3-319-46448-0_2.

[3] LIN T Y, GOYAL P, GIRSHICK R, et al. Focal loss for dense object detection[C/OL]// Proceedings of the IEEE International Conference on Computer Vision. 2017: 2980-2988. DOI: 10.1109/ICCV.2017.324.

[4] BOCHKOVSKIY A, WANG C Y, LIAO H Y M. YOLOv4: Optimal speed and accuracy of object detection[A/OL]. 2020.

[5] JOCHER G, QIU J. Ultralytics YOLO11[EB/OL]. 2024. https://github.com/ultralytics/ultralytics.

[6] KUZNETSOVA A, ROM H, ALLDRIN N, et al. The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale[J/OL]. International Journal of Computer Vision, 2020, 128(7): 1956-1981. DOI: 10.1007/s11263-020-01316-z.

[7] Google. Open Images Dataset V7 and extensions[EB/OL]. 2022. https://storage.googleapis.com/openimages/web/index.html.

[8] LIN T Y, MAIRE M, BELONGIE S, et al. Microsoft COCO: Common objects in context[C/OL]// Computer Vision – ECCV 2014. 2014: 740-755. DOI: 10.1007/978-3-319-10602-1_48.

[9] Ultralytics. Model validation with Ultralytics YOLO[EB/OL]. 2026. https://docs.ultralytics.com/modes/val/.

[10] Open Robotics. ROS 2 Humble documentation: Topics[EB/OL]. 2026. https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html.

[11] Ultralytics. Object detection datasets overview[EB/OL]. 2026. https://docs.ultralytics.com/datasets/detect/.

[12] NVIDIA Corporation. How TensorRT works[EB/OL]. 2026. https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html.

[13] NVIDIA Corporation. TensorRT engine compatibility[EB/OL]. 2026. https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/engine-compatibility.html.